API Testing with playwright

```
import {test ,expect , request} from '@playwright/test';

test("how to validate the data type " , async function ({request}){

    const response = await request.post("https://petstore.swagger.io/v2/store/order",{
        data:{
  "id": 89,
  "petId": 67,
  "quantity": 56,
  "shipDate": "2026-07-23T16:13:23.905Z",
  "status": "placed",
  "complete": true
}
    });

    const responsebody = await response.json();
    console.log(responsebody);

    var id = responsebody.id;
    console.log(id);
    console.log(responsebody.petId);

    expect(responsebody.id).toBeGreaterThan(50);
    expect( responsebody.petId).toBeLessThan(100);
    expect( responsebody.quantity).toBe(56);
    expect( responsebody.shipDate).toContain("2026-07-23");// array and string
    expect( responsebody.status).toBe("placed");
    expect(responsebody.complete).toBe(true);
})
```

Status code validation

```
  "shipDate": "2026-07-23T16:13:23.905Z",
  "status": "placed",
  "complete": true
}
    });

    const responsebody = await response.json();
    console.log(responsebody);

    var id = responsebody.id;
    console.log(id);
    console.log(responsebody.petId);


    console.log(response.status());
    expect(response.status()).toBe(200);


})
```

How to validate the header

```
import{test , expect} from '@playwright/test';

test("how to use post api " , async function({request}){

    const response = await request.post("https://petstore.swagger.io/v2/store/order" ,{

        data :{
            "id": 56,
  "petId": 6,
  "quantity": 8,
  "shipDate": "2026-07-28T16:09:40.626Z",
  "status": "noida",
  "complete": true
        }
    });

    // how to print the response body
    const responsebody = await response.json();
    console.log(responsebody);
// how to print the status and status code
    console.log("status code " + response.status());
    console.log("status text " + response.statusText());

    // how to print the headers
    const headers = response.headers();

    for(const key in headers){

        console.log(key + " : " +  headers[key]);
    }


})
```

How to validate the specific header

```
import { test, expect } from '@playwright/test';

test('Validate response headers', async ({ request }) => {
  const response = await request.get('https://jsonplaceholder.typicode.com/posts/1');

  expect(response.status()).toBe(200);

  const headers = response.headers();

  expect(headers['content-type']).toContain('application/json');
  expect(headers['cache-control']).toBeDefined();
});
```

How to validate the cookies

```
import { test, expect } from '@playwright/test';

test('Validate cookie attributes', async ({ request }) => {
  // Login request
  await request.post('https://example.com/login', {
    data: {
      username: 'testuser',
      password: 'password'
    }
  });

  // Get all stored cookies
  const { cookies } = await request.storageState();

  // Find the session cookie
  const sessionCookie = cookies.find(
    cookie => cookie.name === 'sessionId'
  );

  // Assertions
  expect(sessionCookie).toBeDefined();
  expect(sessionCookie.domain).toContain('example.com');
  expect(sessionCookie.path).toBe('/');
  expect(sessionCookie.httpOnly).toBe(true);
  expect(sessionCookie.secure).toBe(true);
});
```

If sessionCookie is undefined

Print all cookies to see what's actually being stored:

```
const { cookies } = await request.storageState();

console.log(cookies);
```

Example output:

```
[
  {
    name: 'JSESSIONID',
    value: 'ABC123XYZ',
    domain: 'api.example.com',
    path: '/',
    expires: -1,
    httpOnly: true,
    secure: true,
    sameSite: 'Lax'
```

```
  }
]
```

Then update your test:

```
const sessionCookie = cookies.find(
  cookie => cookie.name === 'JSESSIONID'
);

expect(sessionCookie).toBeDefined();
expect(sessionCookie.httpOnly).toBe(true);
expect(sessionCookie.secure).toBe(true);
```

Validate multiple cookie attributes

```
expect(sessionCookie).toMatchObject({
  name: 'sessionId',
  path: '/',
  httpOnly: true,
  secure: true
});
```

How to validate the time playwright with javascript

```
import { test, expect } from '@playwright/test';

test('Validate response time', async ({ request }) => {
  const start = performance.now();

  const response = await request.get('https://api.example.com/users');

  const end = performance.now();

  const responseTime = end - start;

  console.log(`Response Time: ${responseTime.toFixed(2)} ms`);

  expect(responseTime).toBeLessThan(1500);
});
```

1. Bearer Token Authentication (Most Common)

Pass the token in the Authorization header.

```
import { test, expect } from '@playwright/test';

test('GET with Bearer Token', async ({ request }) => {
  const token = 'your_access_token';

  const response = await request.get('https://api.example.com/users', {
    headers: {
```

```
    Authorization: `Bearer ${token}`
  }
 });

 expect(response.status()).toBe(200);
});
```

---

2. Login First, Then Use the Token

Most real-world APIs require logging in to obtain a token.

Step 1: Login

```
const loginResponse = await request.post('https://api.example.com/login', {
 data: {
  username: 'admin',
  password: 'password123'
 }
});

expect(loginResponse.ok()).toBeTruthy();

const loginBody = await loginResponse.json();

const token = loginBody.token;
```

Step 2: Use the token

```
const response = await request.get('https://api.example.com/users', {
 headers: {
  Authorization: `Bearer ${token}`
 }
});

expect(response.status()).toBe(200);
```

---

3. Create an Authenticated API Context (Recommended)

If every API uses the same token, create a request context once.

```
import { request } from '@playwright/test';

const apiContext = await request.newContext({
 baseURL: 'https://api.example.com',
 extraHTTPHeaders: {
  Authorization: 'Bearer your_token'
 }
});

const response = await apiContext.get('/users');
```

This avoids repeating the Authorization header in every request.

---

4. Basic Authentication

```
const response = await request.get('https://api.example.com/users', {
  headers: {
    Authorization: 'Basic ' + Buffer.from('username:password').toString('base64')
  }
});
```

---

5. API Key Authentication

In a header

```
const response = await request.get('https://api.example.com/users', {
  headers: {
    'x-api-key': '123456789'
  }
});
```

As a query parameter

```
const response = await request.get(
  'https://api.example.com/users?apiKey=123456789'
);
```

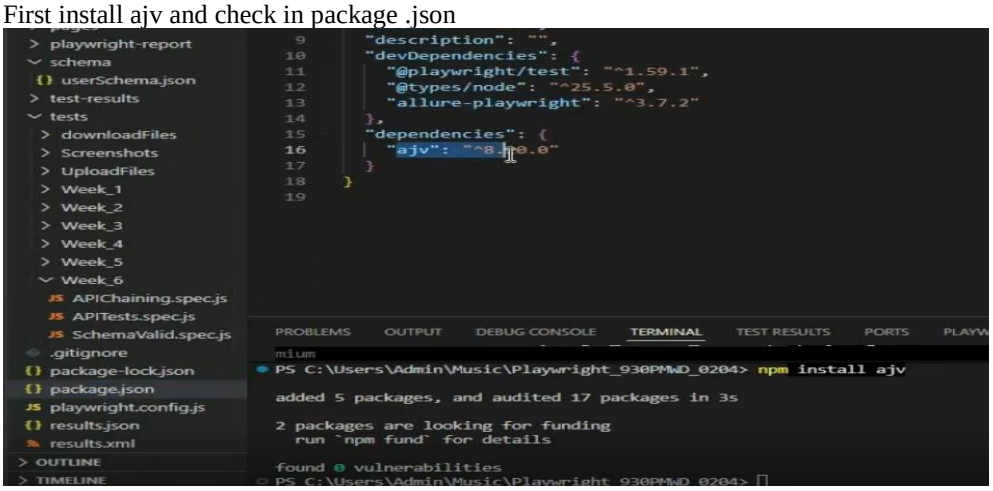
---

6. Cookie Authentication

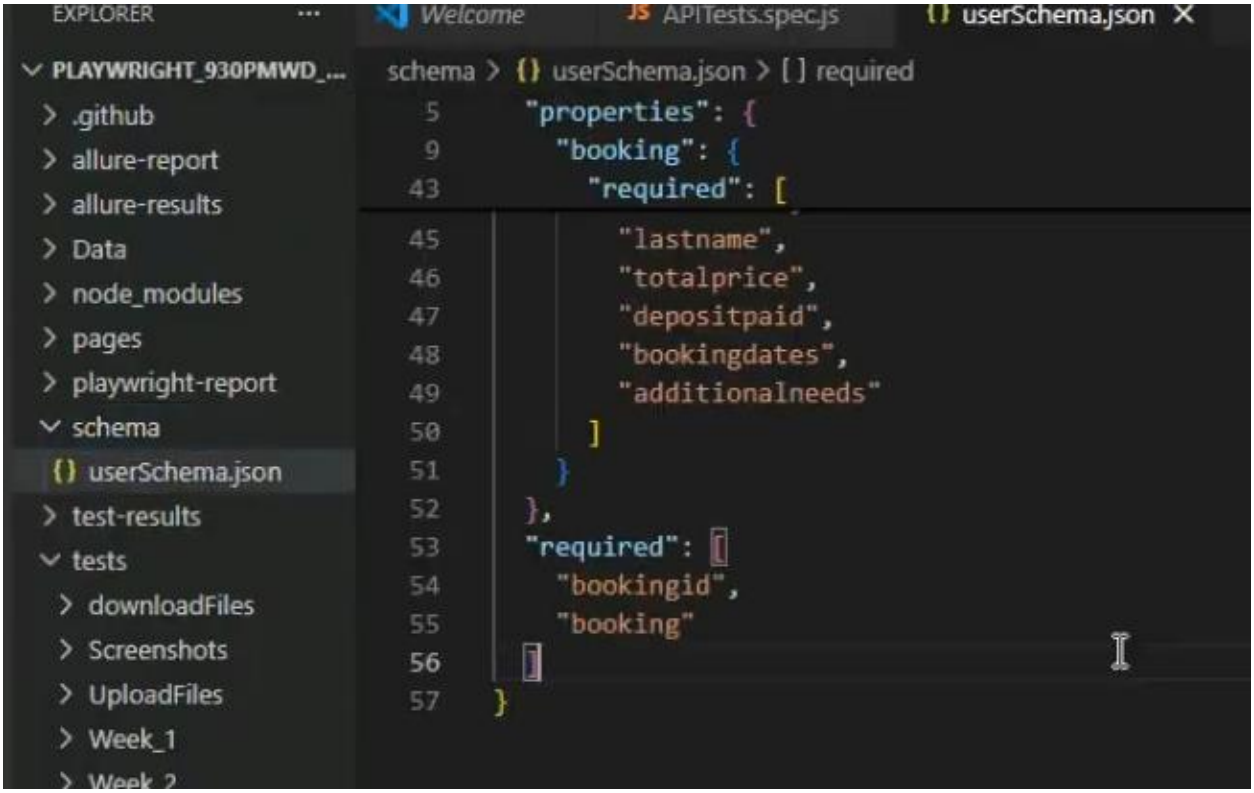
```
const response = await request.get('https://api.example.com/users', {
  headers: {
    Cookie: 'JSESSIONID=ABC123XYZ'
  }
});
```

How to validate the schema

First install ajv and check in package .json

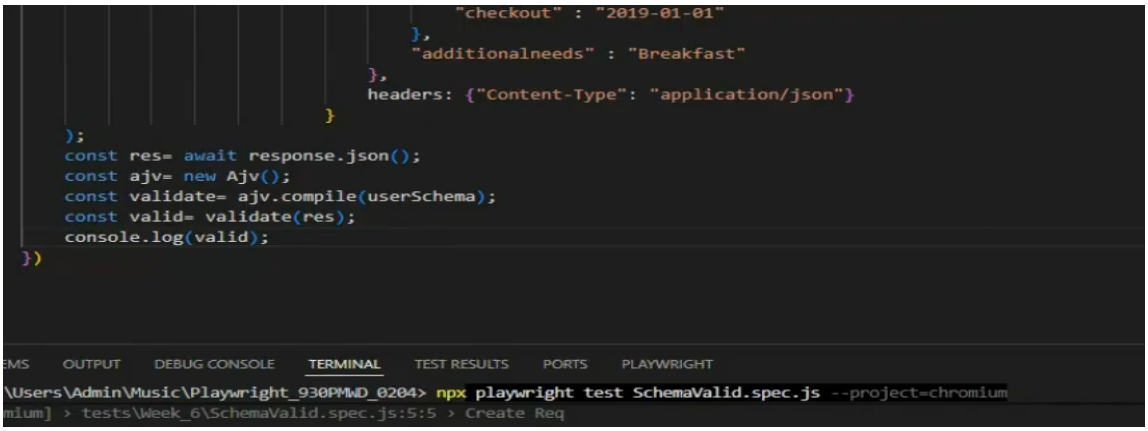


Create a schema folder and save the schema file



Import the file and write the code




"checkout" : "2019-01-01"
},
"additionalneeds" : "Breakfast"
},
headers: {"Content-Type": "application/json"}
}
);
const res= await response.json();
const ajv= new Ajv();
const validate= ajv.compile(userSchema);
const valid= validate(res);
console.log(valid);
})
EMS
OUTPUT
DEBUG CONSOLE
TERMINAL
TEST RESULTS
PORTS
PLAYWRIGHT
\Users\Admin\Music\Playwright_930PMWD_0204> npx playwright test SchemaValid.spec.js --project=chromium
mium] › tests\Week_6\SchemaValid.spec.js:5:5 › Create Req